

Manual Completo — App Controle de Treino

Este manual tem duas partes:

- **Parte 1 — Manual do Usuário:** como usar cada tela do aplicativo.
- **Parte 2 — Manual Técnico:** como o código é organizado, como rodar o projeto e como gerar o APK.

Sumário

Parte 1 — Manual do Usuário

1. Visão geral
2. Primeiro uso
3. Aba “Exercícios”
4. Aba “Peso & Nutrição”
5. Aba “Progresso”
6. Configurações
7. Onde ficam seus dados, e quando são apagados

Parte 2 — Manual Técnico

1. Stack tecnológico
 2. Estrutura de pastas
 3. Modelo de dados
 4. Camada de armazenamento — o padrão `StorageAdapter`
 5. `App.tsx` — o orquestrador
 6. `utils/native.ts` — a fronteira com o nativo
 7. Componentes de UI
 8. Como rodar o projeto localmente
 9. Como gerar o APK
 10. Scripts disponíveis
 11. Solução de problemas comuns no Android Studio
 12. Convenções observadas no código
-

Parte 1 — Manual do Usuário

1. Visão geral

O app é um diário pessoal de treino, feito para rodar **inteiramente offline, no próprio celular**. Ele acompanha três coisas, todas cruzadas entre si:

- **Exercícios** — o que você treina e a evolução de carga em cada um.

- **Peso corporal** — sua evolução de peso ao longo do tempo.
- **Nutrição** — totais diários de calorias e proteína (opcional, resumido, sem diário de refeições).

Não há login, conta de usuário, nuvem ou internet necessária — tudo fica salvo só no seu aparelho.

2. Primeiro uso

Na primeira vez que o app abre, aparece uma tela de splash animada seguida de uma tela de boas-vindas. Nas vezes seguintes, o app pula direto para a tela principal.

3. Aba “Exercícios”

É a tela inicial do app.

Cadastrar um exercício: toque em “Adicionar Exercício”, informe nome e categoria (ex: “Supino Reto” / “Peito”).

Cada card de exercício mostra:

- Nome e categoria (etiqueta abaixo do nome).
- Data do último treino registrado.
- Botão de **registrar treino** (peso, repetições, séries, e observações opcionais).
- Botão de **cronômetro de descanso** entre séries.
- Histórico dos registros daquele exercício (editar/excluir cada um).
- Botão de editar/excluir o próprio exercício.

Buscar e ordenar: no topo da lista há um campo de busca (filtra por nome ou categoria) e um seletor de ordenação:

- Categoria (agrupamento alfabético)
- Nome (A-Z / Z-A)
- Mais recente (pelo último treino registrado)
- Mais registrado (quem tem mais lançamentos)

Grid responsivo: em telas maiores (tablets, por exemplo), os cards se organizam automaticamente em mais colunas para aproveitar o espaço.

4. Aba “Peso & Nutrição”

Reúne dois acompanhamentos na mesma tela, um abaixo do outro.

Peso corporal

- Botão “Registrar” para lançar peso do dia (kg) + data + observação opcional.
- Cartões de “Peso Atual” e “Variação Total”.
- Gráfico de evolução do peso.
- Histórico com editar/excluir cada registro.

Nutrição

Pensada para ser rápida: **um único registro por dia**, sem precisar detalhar cada refeição.

- Botão “Registrar” para lançar **Calorias (kcal)** e/ou **Proteína (g)** do dia, com data e observação opcional. Ambos os campos são opcionais — você pode preencher só um deles.
- Cartões de média de calorias e média de proteína.
- Gráfico com as duas curvas (calorias e proteína) juntas, para facilitar visualizar ao lado da evolução de peso logo acima.
- Histórico com editar/excluir cada registro.

Por design, esse recurso **não é um contador de calorias completo** (não tem busca de alimentos nem base de dados nutricional). A ideia é registrar só o total do dia — já calculado por você em outro lugar, se necessário — para cruzar visualmente com o peso e o treino.

5. Aba “Progresso”

Mostra a evolução detalhada de um exercício específico, escolhido num seletor:

- Peso máximo já levantado.
- Evolução (variação entre o primeiro e o último registro, em kg e %).
- Peso médio.
- Total de treinos registrados.
- Gráfico de evolução de carga ao longo do tempo.
- Gráfico de volume total de treino (peso × séries × repetições).

6. Configurações

Acessível pelo ícone de engrenagem no topo da tela.

Lembrete diário: ativa uma notificação diária, em horário configurável, lembrando de treinar. Só funciona no app instalado (Android/iOS) — não funciona se aberto num navegador comum.

Backup dos dados:

- **Exportar** — gera um arquivo .json com todos os seus dados (exercícios, pesos, nutrição) e abre a tela de compartilhamento do celular, para você salvar onde quiser (Google Drive, e-mail, etc.).
- **Importar** — repõe os dados a partir de um arquivo .json exportado anteriormente.

Tema claro/escuro: alternável pelo ícone de sol/lua no topo da tela.

7. Onde ficam seus dados, e quando são apagados

Os dados ficam salvos **só no armazenamento privado do app, no seu aparelho** (não em nenhum servidor ou nuvem). Eles são apagados se:

- Você **desinstalar o app** (o Android remove automaticamente todo o armazenamento do app).
- Você acessar **Configurações do Android** → **Apps** → **[este app]** → **Armazenamento** → **“Limpar armazenamento/dados”**.
- Você fizer um **reset de fábrica** do celular.

Eles **não são apagados** ao fechar o app, reiniciar o celular, ou atualizar para uma nova versão do APK (desde que o identificador do app não mude).

Recomendação: use o recurso de **Exportar** em Configurações regularmente, principalmente antes de trocar de aparelho ou desinstalar o app — é a única forma de não perder o histórico.

Parte 2 — Manual Técnico

1. Stack tecnológico

Camada	Tecnologia
UI	React 18 + TypeScript
Estilo	Tailwind CSS v4
Componentes de UI	shadcn/ui (Radix UI por baixo)
Gráficos	Recharts
Ícones	lucide-react
Empacotamento web	Vite 6
Empacotamento mobile	Capacitor 7 (gera o projeto Android nativo)
Persistência	localStorage do navegador/WebView

O app **não tem backend**. Toda a “camada de dados” é local ao dispositivo.

2. Estrutura de pastas (dentro de src/)

```

src/
├── App.tsx                # Componente raiz: estado global,
orquestra tudo
├── types.ts               # Definições de tipos (Exercise,
BodyWeightEntry, NutritionEntry, ...)
├── main.tsx               # Ponto de entrada do React
├── index.css              # Estilos globais + tokens do Tailwind
├── components/
│   └── ExerciseList.tsx   # Grid + busca + ordenação de
exercícios
│   ├── ExerciseCard.tsx  # Card individual de um exercício
│   ├── AddExerciseDialog.tsx # Formulário: novo/editar exercício
│   └── AddLogDialog.tsx   # Formulário: novo/editar registro de

```

```

treino
├── ExerciseHistoryDialog.tsx
├── RestTimerDialog.tsx      # Cronômetro de descanso entre séries
├── BodyWeightView.tsx       # Tela de peso corporal (gráfico +
histórico)
├── AddBodyWeightDialog.tsx
├── NutritionView.tsx        # Tela de nutrição (gráfico +
histórico)
├── AddNutritionDialog.tsx
├── ProgressView.tsx         # Gráficos de evolução por exercício
├── SettingsDialog.tsx       # Tema, lembretes, backup
├── WelcomeScreen.tsx / LoadingScreen.tsx
├── ui/                       # Componentes shadcn/ui reutilizáveis
(Button, Card, Dialog, Select...)
├── services/
│   └── storage/
│       └── types.ts          # Interface StorageAdapter
(contrato de persistência)
│       └── localStorageAdapter.ts # Implementação usando
localStorage
│       └── index.ts          # Exporta a instância usada pelo
app
├── utils/
│   ├── date.ts              # Formatação e comparação de datas (formato ISO
'YYYY-MM-DD')
│   └── native.ts             # Única camada que fala com plugins nativos do
Capacitor

```

3. Modelo de dados (src/types.ts)

```

interface WorkoutLog {
  id: string; exerciseId: string;
  weight: number; reps: number; sets: number;
  date: string; notes?: string;
}

interface Exercise {
  id: string; name: string; category: string;
  logs: WorkoutLog[];
}

interface BodyWeightEntry {
  id: string; weight: number; date: string; notes?: string;
}

interface NutritionEntry {
  id: string; date: string;
  calories?: number; protein?: number; notes?: string;
}

type Theme = 'light' | 'dark';

```

```
interface ReminderSettings {
  enabled: boolean; hour: number; minute: number;
}
```

Todas as datas usam o formato YYYY-MM-DD — isso permite comparar/ordenar datas por comparação de string, sem precisar criar objetos Date (evita bugs de fuso horário). Ver `compareDatesDesc` em `utils/date.ts`.

4. Camada de armazenamento — o padrão `StorageAdapter`

A UI **nunca** fala com `localStorage` diretamente. Toda leitura/escrita passa pela interface `StorageAdapter` (`services/storage/types.ts`):

```
interface StorageAdapter {
  loadExercises(): Promise<Exercise[] | null>;
  saveExercises(exercises: Exercise[]): Promise<void>;
  loadBodyWeightLogs(): Promise<BodyWeightEntry[] | null>;
  saveBodyWeightLogs(logs: BodyWeightEntry[]): Promise<void>;
  loadNutritionLogs(): Promise<NutritionEntry[] | null>;
  saveNutritionLogs(logs: NutritionEntry[]): Promise<void>;
  hasVisited(): Promise<boolean>;
  markVisited(): Promise<void>;
  loadTheme(): Promise<Theme | null>;
  saveTheme(theme: Theme): Promise<void>;
  loadReminderSettings(): Promise<ReminderSettings | null>;
  saveReminderSettings(settings: ReminderSettings): Promise<void>;
}
```

Hoje existe uma única implementação, `LocalStorageAdapter`, que grava tudo como JSON em chaves do `localStorage` (`workout-exercises`, `body-weight-logs`, `nutrition-logs`, `theme`, `reminder-settings`, `has-visited`).

Por que isso importa: se um dia for necessário trocar para uma API remota, Supabase, Firebase etc., basta escrever uma nova classe que implemente `StorageAdapter` e trocar a instância exportada em `services/storage/index.ts` — nenhum componente de UI precisa mudar.

Todos os métodos são `async` de propósito (mesmo sendo síncronos hoje), justamente para essa futura substituição ser transparente.

5. `App.tsx` — o orquestrador

É o único componente com estado “de verdade” (`useState` para `exercises`, `bodyWeightLogs`, `nutritionLogs`, `theme`, `reminderSettings` etc.). Os componentes de tela (`ExerciseList`, `BodyWeightView`, `NutritionView`, `ProgressView`) são “burros”: recebem dados e funções de callback via props, e não acessam storage diretamente.

Fluxo de dados:

1. No primeiro `useEffect`, carrega tudo do `storage` em paralelo (`Promise.all`) e popula o estado.
2. Um `useEffect` por tipo de dado persiste automaticamente no `storage` sempre que aquele estado muda (protegido por uma `flag dataLoaded`, para não sobrescrever os dados salvos com o estado vazio do primeiro render).
3. Funções como `addExercise`, `addBodyWeight`, `addNutrition` etc. atualizam o estado local — o `useEffect` de persistência cuida de salvar.

Backup (exportar/importar): `handleExport` monta um objeto `{ exercises, bodyWeightLogs, nutritionLogs, exportedAt }` e delega para `exportTextFile` (em `utils/native.ts`) gerar/compartilhar o arquivo `.json`. `handleImport` faz o caminho inverso, validando que cada campo é um array antes de aplicar ao estado.

6. `utils/native.ts` — a fronteira com o nativo

Este é o **único arquivo** que importa plugins do Capacitor diretamente (`StatusBar`, `Haptics`, `LocalNotifications`, `Filesystem`, `Share`, etc.). Cada função:

- Verifica `Capacitor.isNativePlatform()` primeiro — se estiver rodando num navegador comum, vira um no-op.
- Envolve a chamada nativa em `try/catch`, nunca deixando uma falha de plugin quebrar a aplicação.

Isso mantém o resto do código desacoplado de detalhes de plataforma — os componentes chamam funções como `hapticImpactLight()` sem saber (nem precisar saber) se estão num Android real ou num navegador.

7. Componentes de UI (`components/ui/`)

Vêm do padrão **shadcn/ui**: não é uma biblioteca instalada via npm, são componentes React copiados para dentro do projeto (por isso ficam em `src/`, editáveis). Usam Radix UI por baixo (acessibilidade) e classes utilitárias do Tailwind por cima.

8. Como rodar o projeto localmente (modo web, no navegador)

```
npm install
npm run dev
```

Abre um servidor local (Vite) — útil para desenvolver a UI rapidamente, mas **sem** os recursos nativos (haptics, notificações, compartilhamento de arquivo vira download comum).

9. Como gerar o APK (build para Android)

```
npm install
npm run build           # compila o React para a pasta build/
npx cap sync android    # copia o build para dentro do projeto android/
```

Depois:

1. Abra a pasta `android/` no **Android Studio**.
2. Aguarde a sincronização do Gradle terminar.

3. Build → Build Bundle(s)/APK(s) → Build APK(s).
4. O APK sai em android/app/build/outputs/apk/debug/app-debug.apk.

Também existe o atalho `npm run cap:android`, que faz build + sync + abre o Android Studio automaticamente (`npx cap open android`).

10. Scripts disponíveis (package.json)

Script	O que faz
<code>npm run dev</code>	Sobe o servidor de desenvolvimento (Vite)
<code>npm run build</code>	Gera o build de produção em build/
<code>npm run type-check</code>	Roda <code>tsc --noEmit</code> (só checa tipos, não gera arquivos)
<code>npm run cap:sync</code>	Build + <code>npx cap sync</code> (ambas plataformas)
<code>npm run cap:android</code>	Build + sync + abre o Android Studio
<code>npm run cap:ios</code>	Build + sync + abre o Xcode

11. Solução de problemas comuns no Android Studio

“Please Select Gradle JVM to Import Project” / “Incompatible Gradle JVM version”

Acontece quando o Android Studio usa uma versão do Java (JVM) mais nova do que a versão do Gradle do projeto suporta (Gradle 8.11.1 aqui exige JVM entre 8 e 23). Solução: escolher “Use JVM 21” quando solicitado, ou clicar em “Apply compatible Gradle JDK configuration and sync”. Se o erro voltar mesmo depois disso, apague a pasta de cache android/.gradle (não confundir com android/gradle, que é o wrapper) e sincronize de novo — ela pode guardar um caminho de JDK desatualizado em .gradle/config.properties.

“Error: Module not specified” ao tentar rodar Geralmente indica que o Gradle ainda não sincronizou completamente. Force uma sincronização (File → Sync Project with Gradle Files) e, se persistir, selecione o módulo manualmente em Run → Edit Configurations → Module.

Sincronização travada ou com erros estranhos Último recurso: File → Invalidate Caches... → Invalidate and Restart.

12. Convenções observadas no código

- Comentários e nomes de UI em **português**; nomes de variáveis/funções em **inglês** (padrão do projeto original).
- Toda tela de listagem segue o mesmo padrão: estado vazio dedicado → cartões de resumo → gráfico (se houver dados suficientes) → histórico com editar/excluir → diálogo de formulário reutilizado tanto para criar quanto editar (`entryToEdit` opcional).
- Confirmação de exclusão sempre via `AlertDialog`, nunca exclusão direta no clique.
- Toda ordenação de datas usa `compareDatesDesc` (comparação de string, não Date).